Integrating a Zoho Creator application with another Zoho service using connections and APIs

Insurance Claim Request Form (Basic Fields Use Case) — Zoho Calendar Integration

Sowbarniga A
241401104

1. Aim

To integrate a Zoho Creator insurance claim request form with Zoho Calendar using microservice connections, conditional workflows, and Deluge scripting, so that a claim verification callback is scheduled automatically for every submitted claim.

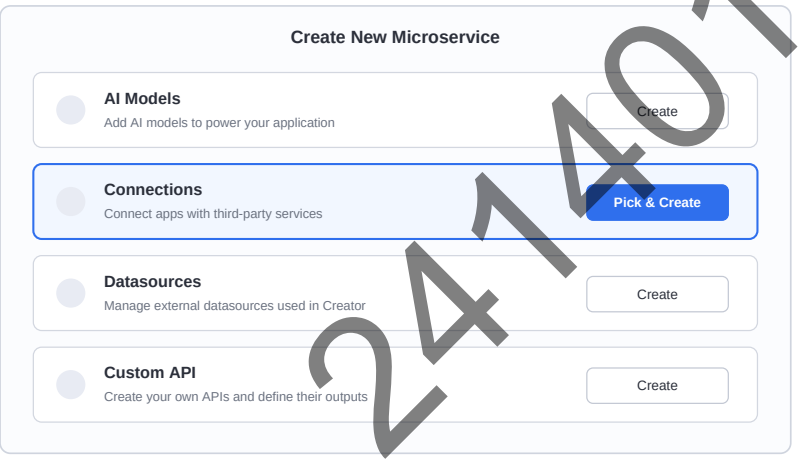
2. Procedure

Phase A: Navigating to the Connection Panel

1. Log in to your Zoho Creator developer account and open the active Insurance Claim Request & Processing Management application.

2. In the left navigation panel under the Develop section, click on Microservices to access the centralized integration environment.

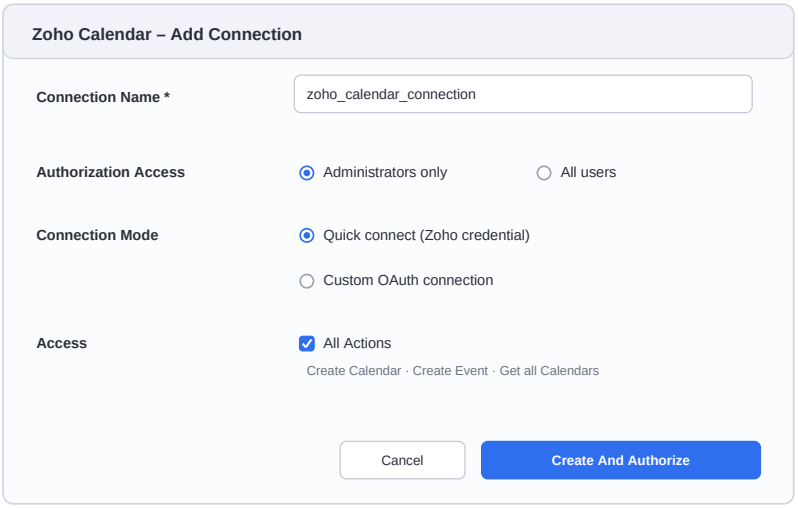
3. Review the available options and choose Connections (Pick & Create) to configure the Zoho Calendar service bridge.



241401104

The Create New Microservice panel lists AI Models, Connections, Datasources, and Custom API; the Connections option (Connect apps with third-party services) is selected using Pick & Create.

4. In the Zoho Calendar – Add Connection dialog, enter the connection name zoho_calendar_connection, set Authorization Access to Administrators only, choose Quick connect (Zoho credential), enable access for All Actions, and click Create And Authorize.



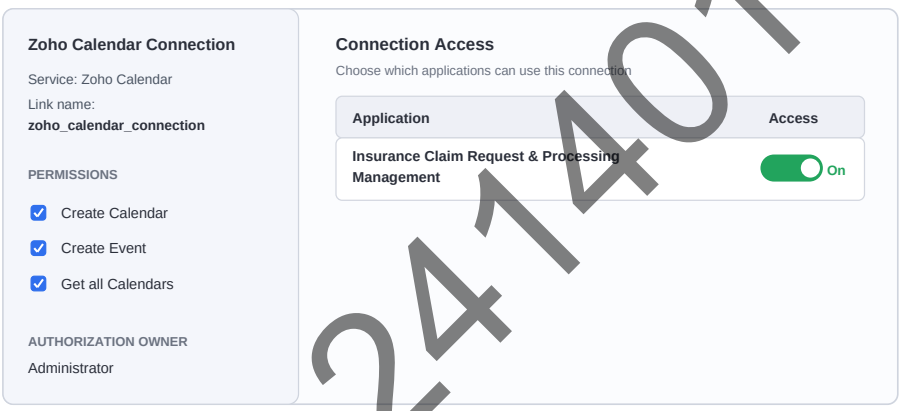
The Zoho Calendar – Add Connection dialog with the connection name zoho_calendar_connection, Authorization Access set to Administrators only, Connection Mode set to Quick connect (Zoho credential), and access enabled for All Actions, followed by Create And Authorize.

Phase B: Verifying Scopes and Application References

5. Click directly on the Zoho Calendar Connection asset to open its security and permission settings dashboard.

6. In the left panel, inspect the assigned operational permissions to confirm API access capabilities (Create Calendar, Create Event, Get all Calendars).

7. In the main pane, verify that the application link for the Insurance Claim Request & Processing Management application is explicitly switched On under the Connection Access matrix.



241401104

The Zoho Calendar Connection management panel showing the service name (Zoho Calendar), link name zoho_calendar_connection, the granted permissions, the authorization owner (Administrator), and the application toggle set to On under Connection Access.

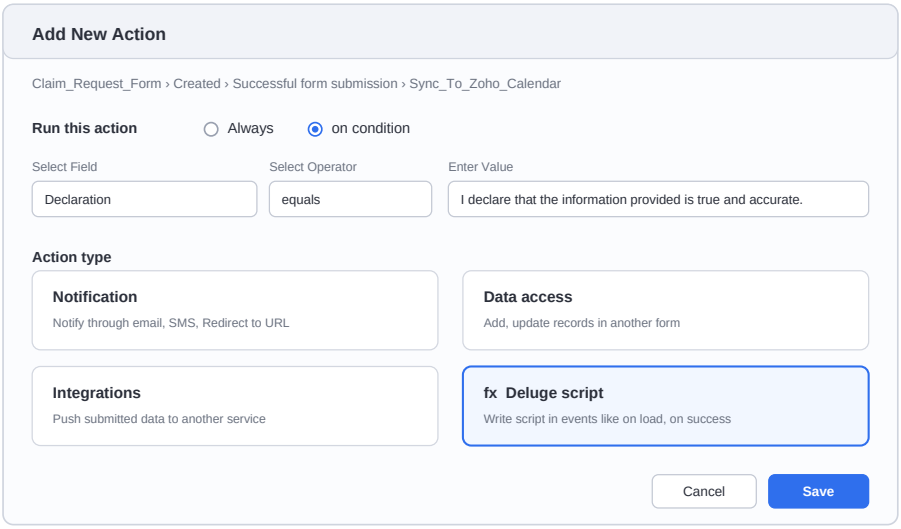
Phase C: Setting Up Conditional Rules

8. Navigate back to the core application, select the Claim Request Form, and click on the Workflow tab in the top header.

9. Create a new form workflow named Sync_To_Zoho_Calendar structured to execute: Created > Successful form submission.

10. Inside the action builder, switch the run parameter from Always to on condition.

11. Open the field mapping tools to link the conditional gate. Choose the Declaration field, set the comparison operator to equals, and bind the value to the checkbox choice: 'I declare that the information provided is true and accurate.'



The Add New Action panel for the Sync_To_Zoho_Calendar workflow (Created > Successful form submission), with Run this action set to on condition, the Select Field / Select Operator / Enter Value condition builder filled in, and the available action types: Notification, Data access, Integrations, and Deluge script.

Phase D: Compiling the Deluge Script Code

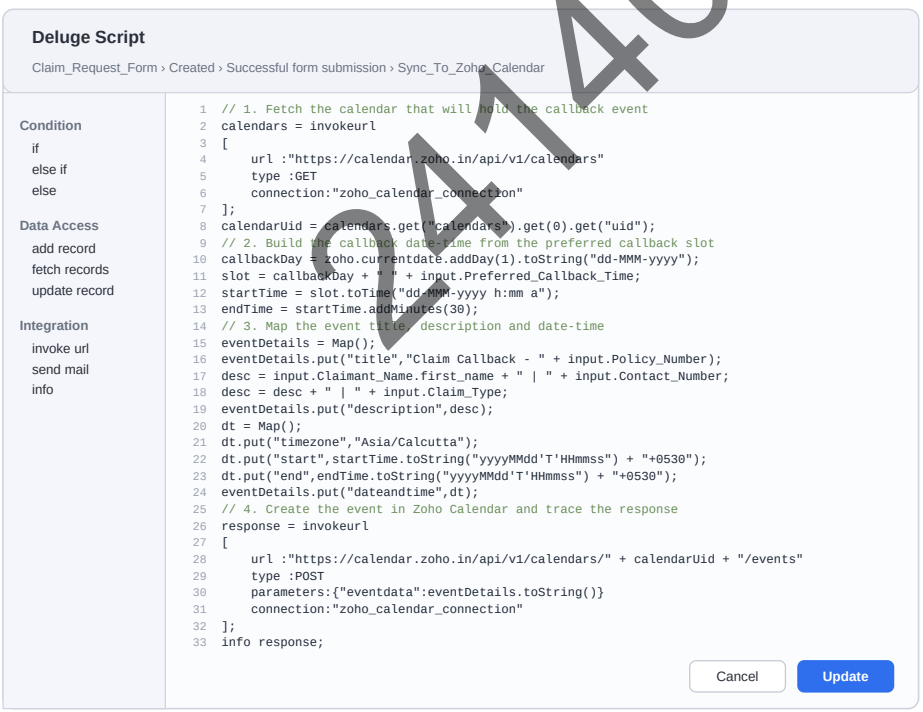
12. Select Deluge script (fx) from the Action type grid to open the embedded Deluge editor workspace.

13. Remove the template elements and insert the data map using the validated field tokens (input.Policy_Number and input.Claim_Type), along with the claimant name, contact number, and preferred callback time fields.

14. Input the structured invokeurl task block targeting the Zoho Calendar API and reference the connection link name (zoho_calendar_connection).

15. Click the blue Update button in the bottom-right corner to run the compiler validation and save the script.

241401104

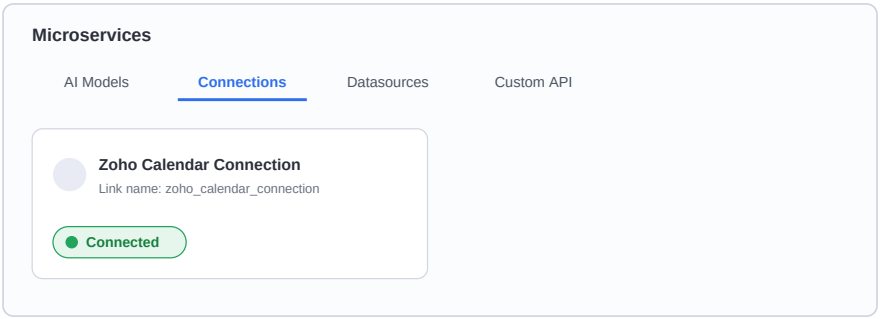


The Deluge Script editor for Sync_To_Zoho_Calendar. The script fetches the calendar with invokeurl (GET), builds the callback date-time from the preferred callback slot, initializes an eventDetails map with the title (policy number), description (claimant, contact and claim type) and date-time, calls invokeurl with type POST using the connection zoho_calendar_connection, and traces the response using info.

3. Output & System Verification

The end-to-end integration architecture was successfully deployed within the application ecosystem. As verified in the main microservices interface, the Zoho Calendar API pipeline is established and operational.

- **Connection Diagnostic Result:** Connected (Green Status Verified)
- **Operational Availability:** The connection link name zoho_calendar_connection is fully compiled and active within the form's logic flow.
- **Workflow Status:** Sync_To_Zoho_Calendar was saved without compilation errors and is mapped to the Claim Request Form (Created > Successful form submission).



The Microservices dashboard (Connections tab) showing the Zoho Calendar Connection card with the link name zoho_calendar_connection and a green Connected status, confirming the integration bridge is established.

4. Conclusion

The lab exercise successfully demonstrates how to connect a low-code application to external cloud services using microservices, conditional workflows, and custom Deluge scripting. By binding the Claim Request Form criteria to the zoho_calendar_connection module, the project establishes a working proof-of-concept for automated claim verification scheduling, where every declared claim request blocks a callback slot on the claims team's master calendar at the claimant's preferred time.

241401104